



**BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

GUILHERME FABRICIO BRITO DA ROSA  
HARRISON DE CARVALHO ALVARENGA  
RAUL SOARES DE CARVALHO

**TRABALHO COMPILADORES**

**Análise Léxica e Sintática**

LAVRAS

2025

## 1. Introdução

A construção de compiladores é um processo fundamental na ciência da computação, permitindo a tradução de código escrito em linguagens de alto nível para código de máquina executável. A primeira fase desse processo é a análise léxica, responsável por ler o fluxo de caracteres do código-fonte e agrupá-los em unidades significativas chamadas tokens.

Este relatório descreve a implementação de um analisador léxico para uma linguagem de programação hipotética denominada C-, utilizando a ferramenta Flex (Fast Lexical Analyzer Generator). O objetivo principal foi criar um analisador capaz de reconhecer os diferentes componentes léxicos da linguagem C- (palavras-chave, identificadores, números, operadores, pontuação), ignorar espaços em branco e comentários, e reportar erros léxicos encontrados, indicando a linha e coluna onde ocorreram.

A segunda fase desse processo é a análise sintática, cuja função é verificar se a sequência de tokens produzida pela análise léxica forma uma estrutura gramaticalmente válida segundo as regras da linguagem C-. Nesta etapa, os tokens são organizados em uma árvore de derivação (ou árvore sintática), refletindo a estrutura hierárquica das construções do programa, como comandos, expressões e declarações.

É apresentada a implementação de um analisador sintático para a linguagem C-, utilizando a ferramenta Bison (um gerador de analisadores sintáticos compatível com o Yacc). O analisador sintático foi desenvolvido com base em uma gramática definida para a linguagem, sendo capaz de identificar construções corretas, detectar erros de sintaxe e reportá-los de forma clara ao usuário, também indicando a linha e a coluna onde o erro ocorreu.

Essa etapa é essencial para garantir que o código-fonte tenha uma estrutura coerente antes de ser traduzido para uma representação intermediária ou diretamente para código de máquina.

## 2. Referencial Teórico

### 2.1 Análise Léxica

A análise léxica é a fase inicial do processo de compilação. O analisador léxico, também conhecido como scanner, lê o código-fonte caractere por caractere e o divide em uma sequência de tokens. Cada token representa uma unidade léxica da linguagem, como uma palavra-chave (if, while), um identificador (variável, func), um número (123, 3.14), ou um operador (+, =). Lexema: Sequência de caracteres no código-fonte que corresponde a um padrão de um token. Por exemplo, if, contador, 100, +. Token: Par composto por um nome (ou tipo) de token e, opcionalmente, um valor de atributo. Exemplos: (KEYWORD, "if"), (IDENTIFIER, "contador"), (INTEGER, 100), (OPERATOR, "+"). Padrão: Descrição da forma que os lexemas de um token podem assumir, geralmente especificada por expressões regulares.

### 2.2 Expressões Regulares

Expressões regulares são uma notação formal para especificar padrões de strings. Elas são amplamente utilizadas para definir os padrões dos tokens em análise léxica. Operações comuns incluem concatenação, união (|), e fecho de Kleene (\*, zero ou mais ocorrências; +, uma ou mais ocorrências; ?, zero ou uma ocorrência).

## 2.3 Diagrama de Transição

Foram feitos dois diagramas de transição: o primeiro (Automato\_Completo.jff.jpg) contém um único autômato com todas as transições, porém com baixa organização visual devido a várias transições que incidem em um mesmo estado. O segundo (Automato\_Desmembrado.jff.jpg) contém nove autômatos, mas que representam apenas um, de tal modo que podemos observar dois nomes de estado repetindo neles (q0 representa o mesmo estado inicial, e q27 representa o mesmo estado). Também há um estado adicional (q13). Essas diferenças são apenas para melhorar a organização visual, visando o melhor entendimento do diagrama. A ferramenta utilizada para criar o diagrama foi o JFLAP.

## 2.4 Flex (Fast Lexical Analyzer Generator)

Flex é uma ferramenta que gera analisadores léxicos (scanners) em linguagem C a partir de um arquivo de entrada (.l) que descreve os padrões dos tokens usando expressões regulares e as ações C a serem executadas quando cada padrão é reconhecido. A estrutura básica de um arquivo .l é:

```
C/C++
%{
    // Definições C (includes, variáveis globais)
}%
/* Definições de nomes para expressões regulares */
%%
/* Regras: Padrão { Ação C } */
%%
/* Código C adicional (funções auxiliares, main) */
```

## 2.5 Análise sintática

A análise sintática é a segunda etapa do processo de compilação e tem como objetivo verificar se a sequência de tokens, produzida pelo analisador léxico, obedece à estrutura gramatical da linguagem de programação. Essa etapa é realizada por um **analisador**

**sintático** (ou *parser*), que utiliza uma **gramática livre de contexto** (GLC) para guiar a análise.

Durante esse processo, os tokens são agrupados em estruturas hierárquicas que representam as construções da linguagem, como expressões, comandos e declarações. O resultado pode ser uma **árvore sintática** (ou *parse tree*), que descreve de forma estruturada a sintaxe do código analisado.

Existem dois tipos principais de análise sintática:

- **Análise descendente (top-down)**: constrói a árvore sintática a partir do símbolo inicial, expandindo-a em direção aos símbolos terminais.
- **Análise ascendente (bottom-up)**: parte dos símbolos terminais e tenta reduzir a sequência até o símbolo inicial da gramática.

Erros sintáticos, como parênteses desbalanceados ou estruturas mal formadas, são detectados nesta fase, e o analisador deve ser capaz de reportá-los com clareza, indicando o local e a natureza do erro.

## 2.6 Gramática Livre de Contexto (GLC)

Uma Gramática Livre de Contexto é um conjunto de regras de produção que definem como os símbolos de uma linguagem podem ser combinados para formar sentenças válidas. Ela é composta por:

- Um conjunto de **símbolos terminais** (tokens),
- Um conjunto de **símbolos não-terminais** (variáveis),
- Um símbolo inicial,
- Um conjunto de **produções** (regras) da forma  $A \rightarrow \alpha$ , onde  $A$  é um não-terminal e  $\alpha$  é uma sequência de terminais e/ou não-terminais.

GLCs são fundamentais para a definição da sintaxe de linguagens de programação e são amplamente utilizadas na construção de compiladores.

## 2.7 Bison (GNU Parser Generator)

**Bison** é uma ferramenta que gera analisadores sintáticos em linguagem C a partir de uma especificação de gramática. Ele é compatível com o Yacc (Yet Another Compiler Compiler) e funciona em conjunto com o Flex, que fornece os tokens ao Bison.

Um arquivo **.y** do Bison é dividido em três partes principais:

```

C/C++
%{
    // C declarations
%}
%token IDENTIFIER NUMBER IF ELSE WHILE RETURN
%%
stmt_list: stmt_list stmt | stmt ;
stmt: IF '(' expr ')' stmt ELSE stmt | ...
%%
int main() {
    yyparse();
    return 0;
}

```

O Bison gera uma função `yyparse()` que coordena a análise sintática. Quando encontra uma produção válida, pode executar ações associadas escritas em C. Em caso de erro sintático, permite especificar como o compilador deve se comportar.

### Análise Semântica

A **análise semântica** é a terceira etapa do processo de compilação, sucedendo a análise sintática, e tem como principal objetivo verificar a correção lógica e o significado do código-fonte. Enquanto o analisador sintático se preocupa com a estrutura gramatical, o analisador semântico foca nas regras de coerência e compatibilidade de tipos, garantindo que as operações e declarações na linguagem façam sentido. Essa etapa é crucial para identificar erros que a análise sintática não consegue detectar, como o uso de uma variável não declarada, a atribuição de um valor de um tipo incompatível a uma variável, ou a chamada de uma função com o número incorreto de argumentos.

Durante a análise semântica, o compilador constrói e utiliza **tabelas de símbolos** para armazenar informações sobre identificadores (variáveis, funções, classes, etc.), incluindo seus tipos de dados, escopo e outros atributos relevantes. Essa fase também realiza a **verificação de tipos**, garantindo que as operações sejam aplicadas a dados de tipos compatíveis (por exemplo, não é permitido somar um número a um texto a menos que a linguagem defina explicitamente essa conversão). O resultado da análise semântica é um código sintaticamente e semanticamente correto, pronto para a próxima fase do compilador.

**Erros semânticos**, como a tentativa de dividir um número por zero ou a passagem de um argumento de tipo errado para uma função, são detectados nesta fase. O analisador semântico deve ser capaz de reportar esses erros de forma precisa, auxiliando o programador a identificar e corrigir problemas lógicos no código.

## Tabela de Símbolos

A **tabela de símbolos** é uma estrutura de dados fundamental no processo de compilação, atuando como um banco de dados centralizado que armazena todas as informações essenciais sobre os identificadores (nomes de variáveis, funções, classes, constantes, etc.) presentes no código-fonte. Ela é construída e acessada por diversas fases do compilador, começando na análise léxica (onde identificadores são reconhecidos), passando pela análise sintática, e sendo intensivamente utilizada durante a análise semântica e a geração de código.

A tabela de símbolos não é uma fase isolada do compilador, mas sim um componente vital que **suporta as demais etapas**, permitindo que o compilador mantenha um registro e recupere detalhes importantes sobre cada elemento nomeado do programa.

A principal função da tabela de símbolos é registrar atributos cruciais para cada identificador. Isso inclui, mas não se limita a, seu **tipo de dado** (inteiro, float, string, booleano, etc.), seu **escopo** (onde no programa ele é visível e pode ser acessado, como global, local a uma função ou a um bloco), seu **endereço de memória** ou deslocamento (atribuído nas fases de geração de código), o **número e tipo de parâmetros** se for uma função, e até mesmo se é uma constante ou uma palavra-chave reservada. A forma como essa estrutura é implementada pode variar, utilizando listas ligadas, árvores de busca, tabelas *hash* ou uma combinação delas, sempre buscando eficiência na inserção e recuperação de informações.

Erros relacionados à inconsistência no uso de identificadores, como a **redeclaração de uma variável** no mesmo escopo, o **uso de uma variável não declarada**, ou **invocações de funções com assinaturas incorretas**, são detectados e reportados com base nas informações mantidas na tabela de símbolos. Ela é a memória do compilador sobre o programa, garantindo que as regras de visibilidade e tipagem sejam respeitadas em todo o processo de tradução.

## Gerador de Código Intermediário

O propósito principal do gerador de código intermediário é traduzir o código-fonte, que agora está sintaticamente e semanticamente correto, para uma representação de nível mais baixo, mas ainda independente da arquitetura de máquina final. Essa representação intermediária serve como uma ponte crucial entre o código de alto nível e o código de máquina executável, simplificando as fases subsequentes de otimização e geração de código final. Ao invés de gerar diretamente o código de máquina, o que seria complexo, o código intermediário permite que o compilador seja dividido em partes mais gerenciáveis, facilitando a portabilidade para diferentes plataformas e a aplicação de otimizações. Existem diversas formas de representação de código intermediário, como o **código de três endereços (TAC)**, que expressa cada operação com no máximo três operandos, ou a **forma de quadruplas e triplas**, que organizam as operações de maneira similar.

Independentemente da forma, o código intermediário é projetado para ser fácil de gerar a partir da árvore sintática e fácil de transformar em código de máquina. Durante essa fase, o compilador também pode realizar algumas otimizações de alto nível que são mais eficazes nessa representação, como a eliminação de sub-expressões comuns ou a propagação de constantes. O resultado é uma representação do programa que é mais

próxima da máquina, mas ainda simbólica e otimizações.

Erros lógicos ou de projeto do compilador que resultam em uma representação intermediária inválida podem ser detectados nesta fase. Embora a maioria dos erros do código-fonte já tenha sido identificada nas fases anteriores, problemas como a geração incorreta de sequências de operações ou a manipulação inadequada de endereços podem se manifestar aqui, exigindo que o gerador de código intermediário seja robusto e preciso.

## 2.8 Integração entre Flex e Bison

O Flex e o Bison trabalham juntos para realizar a análise léxica e sintática. O Flex gera uma função chamada `yylex()` que é usada pelo `yyparse()` do Bison para obter os próximos tokens. Essa integração permite que a análise léxica alimente diretamente a análise sintática, resultando em um processo de compilação mais estruturado e eficiente.

Para que isso funcione corretamente:

- Os tokens definidos no Bison devem estar presentes no Flex com os mesmos nomes ou equivalentes,
- O Flex deve retornar os tokens com os nomes definidos por `%token` no Bison,
- Ambos os arquivos devem ser compilados e vinculados corretamente.

## 3. Descrição do Trabalho

O analisador léxico para a linguagem C- foi implementado utilizando a ferramenta Flex, seguindo a estrutura padrão de um arquivo .l.

### 3.1 Estratégias de Solução

a. Definições Globais (`%{ ... %}`):

Nesta seção, foram incluídos os cabeçalhos C necessários (`stdio.h`, `string.h`) e declaradas variáveis globais para rastrear o número da linha (`line_number`, inicializada com 1), o número da coluna (`column_number`, inicializado com 1) e a contagem de erros (`errors_count`). A variável `yyout` foi definida para direcionar a saída para `stdout`.

b. Definições de Expressões Regulares:

Foram definidos nomes para expressões regulares frequentemente usadas, como `digit`, `letter`, `integer_num`, `float_num`, `identifier`, `ws` (whitespace), `newline`, etc. Isso melhora a legibilidade das regras. A expressão para `float_num`

`{digit}+(\.{digit}+)?([Ee][+-]?{digit}+)?` foi definida para reconhecer números com parte decimal e/ou expoente opcionais. A definição `array_identifier` foi criada para reconhecer identificadores seguidos por um índice inteiro entre colchetes.

c. Estado de Comentário (%x COMMENT):

Foi definido um estado exclusivo (COMMENT) para lidar corretamente com comentários de múltiplas linhas (`/* ... */`), garantindo que o conteúdo do comentário seja lido e descartado, sem ser interpretado como tokens, enquanto se atualiza corretamente os números de linha e coluna.

d. Regras Léxicas (%% ... %%):

Esta seção contém o mapeamento entre os padrões (expressões regulares ou strings literais) e as ações C.

- **Comentários:** A regra `/*` ativa o estado COMMENT. Dentro deste estado, regras específicas consomem o conteúdo do comentário, incluindo novas linhas (atualizando `line_number` e resetando `column_number`), até que o padrão `*/` seja encontrado, retornando ao estado inicial (INITIAL). A saída (`fprintf`) dentro das regras de comentário parece ter sido usada para depuração, mostrando o conteúdo do comentário sendo processado.
- **Palavras-Reservadas:** Strings literais como `int`, `float`, `if`, `while`, etc., são reconhecidas diretamente. A ação associada imprime o número da linha, coluna, o lexema (`yytext`) e o tipo do token (ex: PALAVRA RESERVADA int). A coluna é atualizada (`column_number += yyleng`).
- **Operadores e Pontuação:** Strings literais para operadores (`+`, `-`, `*`, `/`, `=`, `==`, `!=`, etc.) e pontuação (`(`, `)`, `{`, `}`, `;`, etc.) são tratadas de forma similar às palavras-chave.
- **Números:** As regras `{float_num}` e `{integer_num}` reconhecem números reais e inteiros, respectivamente, usando as definições prévias. A ação imprime as informações do token e atualiza a coluna. Observação: A ordem destas regras é importante; `{float_num}` deveria idealmente vir antes de `{integer_num}` para garantir o reconhecimento correto de números que podem ser interpretados como ambos (ex: 123.45 poderia ser reconhecido como o inteiro 123 se a regra de inteiro vier primeiro).
- **Identificadores:** A regra `{identifier}` reconhece identificadores válidos. A regra `{array_identifier}` reconhece especificamente identificadores de vetores. Observação: A regra `{letter}` isolada pode causar conflitos com `{identifier}`, pois um identificador começa com uma letra. A regra `{array_identifier}` também pode precisar de ajuste de ordem em relação a `{identifier}`.
- **Espaços em Branco e Novas Linhas:** A regra `{ws}` simplesmente atualiza o número da coluna para ignorar espaços e tabs. A regra para newline (não



explicitamente visível no trecho, mas necessária) deveria incrementar `line_number` e resetar `column_number`.

- **Erros Léxicos:** A regra `{other}` captura qualquer caractere que não corresponda a nenhuma das regras anteriores. Ela imprime uma mensagem de erro indicando o caractere inválido, a linha e a coluna, e incrementa `errors_count`.

e. Código Auxiliar (%% ... %% após regras):

Contém a função `main`, que abre o arquivo de entrada fornecido como argumento (`argv[1]`), atribui o ponteiro do arquivo a `yyin`, chama `yylex()` em loop (implicitamente, pois `yylex()` é chamado até o fim do arquivo) para processar a entrada, e fecha o arquivo. A função `yywrap()` retorna 1, indicando que não há mais arquivos de entrada a processar.

### Estratégias de Solução do Analisador Sintático

a. Variáveis Externas:

As variáveis externas `line_number`, `column_number` e `errors_count` são declaradas e manipuladas no arquivo `Flex`, e seus valores são acessados pelo Bison ao serem declaradas como "extern". Elas indicam, respectivamente, a linha, coluna e erros encontrados no programa até o momento.

b. Diretiva `%token`:

Os itens com a diretiva `%token` representam os terminais de gramática utilizados pelo analisador sintático.

c. Diretiva `%nonassoc`:

Os itens com a diretiva `%nonassoc` representam precedência nas regras da gramática, e são usados para resolver conflitos `shift/reduce` causados pelo problema "dangling else" e pelas regras de erro.

d. Diretiva `%start`:

O item com essa diretiva é a regra inicial da gramática utilizada pelo Bison.

e. Recursão a esquerda:

As regras gramaticais foram escritas com recursão a esquerda pois a ferramenta Bison lida melhor com esse tipo de recursão, uma vez que o Bison mantém um pilha interna com um registro de todos símbolos visto por regras parcialmente analisadas, ou seja, fosse a regra `X` recursiva a direita, teríamos o dobro de entradas na pilha para cada chamada de `X`.

f. Erros, `yyerror` e `yyerrok`:

Os erros são detectados na análise sintática através de produções de erro, permitindo assim identificar erros previstos na gramática e continuar a análise sintática. A função `yyerror` é chamada toda vez que o Bison encontra um erro, e recebe uma mensagem como parâmetro, a qual usamos para descrever o erro, e também aponta onde o erro ocorreu, usando as variáveis externas `line_number` e `column_number`. O macro `yyerrok` limpa o estado de erro que o Bison entra após encontrar um erro.

- g. arranjo\_dimensao e arranjo\_acesso:  
As produções arranjo\_dimensao e arranjo\_acesso foram adicionadas a gramática para facilitar a organização e leitura do código, porém não alteram absolutamente nada da gramática original.
- h. Regras Sintáticas:  
As regras sintáticas seguem as regras especificadas no enunciado sem nenhuma alteração e obedecendo todas as restrições da linguagem C-.

### **Estratégias de Solução do Analisador Sintático, Tabela de Símbolos e Gerador de Código de Três Endereços**

- a. Estrutura de Dados para Símbolos:  
Foi criada uma estrutura SymbolEntry contendo os seguintes campos: char\* name, DataType data\_type, SymbolType symbol\_type, int scope\_level, int line\_declared, int memory\_address, int is\_array, int\* array\_dimensions, struct SymbolEntry\* next, que representam respectivamente, o nome do símbolo, o tipo do dado, tipo de símbolo, o nível de escopo, linha da declaração, endereço de memória, flag indicando se é arranjo, dimensões do arranjo, e uma lista ligada para tratar colisões.
- b. Estrutura de Dados para Código de Três Endereços:  
Foi criada uma estrutura Codeline contendo os seguintes campos: char\* op, char\* arg1, char\* arg2, char\* result, int line\_num e struct Codeline\* next, que representam respectivamente, no contexto do código de três endereços, o operador, o primeiro argumento, o segundo argumento, o resultado, a linha do código e um apontador para o próximo trecho de código.
- c. Tabela de Símbolos com Hash  
A tabela de símbolos utiliza uma hashtable de 211 posições, visando eficiência, e uma função hash é utilizada para inserir os símbolos na tabela. A tabela resolve as colisões utilizando encadeamento separado onde cada elemento da tabela mantém uma lista ligada.
- d. Adição de Regras Semânticas ao Parser:  
O parser foi aprimorado para ter regras semânticas que são utilizadas durante a análise semântica.
- e. Função check\_binary\_op:  
A função check\_binary\_op foi implementada para verificar se os tipos de uma operação são compatíveis, bem como faz a conversão implícita de int para float.
- f. Gerenciamento de Escopo:  
O gerenciamento de escopo é feito usando uma estrutura ScopeStack, com os campos int scope\_level, que representa o nível do escopo, SymbolTable\* table, que aponta para uma tabela de símbolos, e struct ScopeStack\* parent, que aponta para o escopo que precede o escopo atual. As operações enter\_scope(), que empilha um novo escopo, exit\_scope(), que desempilha o escopo atual, e lookup\_symbol(), que realiza uma busca hierárquica. A função check\_variable\_usage verifica a o escopo das variáveis.
- g. Padrões de Tradução:  
Os seguintes padrões de tradução foram adotados:

- Expressões Aritméticas:  
 $E \rightarrow E_1 + E_2 \{$   
 $\quad t = \text{new\_temp}();$   
 $\quad \text{emit}("+", E_1.\text{place}, E_2.\text{place}, t);$   
 $\quad E.\text{place} = t;$   
 $\}$
- Comandos de Atribuição:  
 $S \rightarrow \text{id} = E \{$   
 $\quad \text{emit} "=", E.\text{place}, "", \text{id}.\text{place});$   
 $\}$
- Estruturas de Controle (if):  
 $S \rightarrow \text{if} (E) S_1 \{$   
 $\quad L = \text{new\_label}();$   
 $\quad \text{emit}("if\_false", E.\text{place}, "", L);$   
 $\quad S_1.\text{code};$   
 $\quad \text{emit}("label", "", "", L);$   
 $\}$

h. Gerenciamento de Temporários:  
`##`

## Geração e Compilação

O analisador léxico é gerado executando `flex c-minus-lexer.l`, que produz o arquivo `lex.yy.c`. Este arquivo é então compilado usando `gcc lex.yy.c -lfl -o c-minus-lexer` para criar o executável.

### 3.2 Geração e Compilação

O analisador léxico é gerado executando `flex c-minus-lexer.l`, que produz o arquivo `lex.yy.c`. Este arquivo é então compilado usando `gcc lex.yy.c -lfl -o c-minus-lexer` para criar o executável.

### 3.3 Analisador Sintático

O analisador sintático para a linguagem C- foi desenvolvido com a ferramenta **Bison**, utilizando um arquivo `parser.y` que define a gramática da linguagem e as ações a serem executadas quando as produções são reconhecidas.

O parser implementado utiliza uma **gramática livre de contexto (GLC)** compatível com a especificação da linguagem C-, com produções definidas para expressões, comandos, estruturas de controle (como `if`, `while`) e declarações de variáveis e funções.

Cada produção define regras da linguagem e pode estar associada a uma ação escrita em C. Essas ações são usadas principalmente para depuração ou geração de árvore sintática

(neste trabalho, ações foram usadas para indicar o reconhecimento de construções da linguagem e para exibir mensagens de erro quando apropriado).

### 3.4 Integração entre Analisador Léxico e Sintático

A integração entre o Flex e o Bison foi realizada através do parser, gerado pelo Flex, que retorna tokens ao `yyparse()` gerado pelo Bison. Os tokens reconhecidos no arquivo `lexer.l` devem estar devidamente declarados no arquivo `parser.y` com a diretiva `%token`, e o valor de retorno de cada token no Flex deve corresponder ao esperado pelo Bison.

Por exemplo, no Flex:

```
C/C++

"int"      { return INT; }

[a-zA-Z_][a-zA-Z0-9_]* { yylval.id = strdup(yytext); return
IDENTIFIER; }
```

No Bison:

```
C/C++

%token INT IDENTIFIER
```

O tipo de `yylval` foi declarado para suportar diferentes tipos de atributos, utilizando uma `union` com campos para strings e números, caso necessário.

### 3.5 Compilação do Analisador Sintático

A compilação e execução do analisador sintático foram automatizadas por meio de um script em shell (`.sh`). Esse script é responsável por validar o arquivo de entrada, gerar o analisador sintático e léxico com as ferramentas **Bison** e **Flex**, compilar os arquivos resultantes e executar o analisador com um arquivo de teste.

O processo consiste nas seguintes etapas:

1. **Verificação do arquivo de entrada:**

O script verifica se o nome do arquivo de teste foi passado como argumento. Caso contrário, exibe instruções de uso. Em seguida, procura o arquivo informado tanto no diretório raiz quanto na pasta `tests/parser`.

## 2. Verificação da estrutura de diretórios:

O script garante que os diretórios `c-minus/parser` e `c-minus/lexer` existam e contenham os arquivos `parser.y` e `lexer.l`, respectivamente. Caso algum deles esteja ausente, a execução é interrompida com uma mensagem de erro.

## 3. Geração do analisador sintático:

O comando abaixo utiliza o Bison para gerar os arquivos `parser.tab.c` e `parser.tab.h` a partir do arquivo de gramática `parser.y`:

```
None  
bison -d -v -t -o c-minus/parser/parser.tab.c  
c-minus/parser/parser.y
```

As opções utilizadas têm as seguintes finalidades:

- `-d`: gera o arquivo de cabeçalho `parser.tab.h` (necessário para o Flex).
- `-v`: gera um arquivo `.output` com detalhes da análise da gramática.
- `-t`: ativa a geração de código de rastreamento para depuração.
- `-o`: define o nome do arquivo de saída gerado.

## 4. Geração do analisador léxico:

Com o arquivo `parser.tab.h` já disponível, o Flex é utilizado para gerar o scanner léxico:

```
None  
  
flex -d -o c-minus/lexer/scanner.yy.c  
c-minus/lexer/lexer.l
```

## 5. Compilação dos componentes:

O compilador `gcc` é então utilizado para compilar os arquivos C gerados (`scanner.yy.c` e `parser.tab.c`), vinculando-os com a biblioteca `-lfl`:

None

```
gcc -o tests/parser/c- c-minus/lexer/scanner.yy.c  
c-minus/parser/parser.tab.c -lfl
```

#### 6. Execução do analisador:

O script executa o analisador com o arquivo de teste passado como argumento e salva o log da execução em `tests/parser/log.txt`. A variável de ambiente `YYDEBUG=1` é ativada para permitir o rastreamento detalhado do parser:

None

```
export YYDEBUG=1  
  
./tests/parser/c- nome_arquivo.txt 2>  
tests/parser/log.txt
```

## 4. Testes Executados e Resultados Obtidos

Para validar o analisador léxico, foram criados diversos arquivos de teste (.txt) contendo:

- Código C- válido com diferentes tipos de tokens (palavras-chave, identificadores, números inteiros e reais, operadores, pontuação).
- O código contendo comentários de múltiplas linhas.
- O código contendo espaços em branco e múltiplas linhas.
- O código contendo erros léxicos (caracteres inválidos como @, \$).

O analisador foi executado com cada arquivo de teste usando o script `scriptCompileAndRun.sh` ou manualmente (`./c-minus-lexer teste.txt`).

C/C++

```
/* Calcula fatorial */  
int fact(int n) {  
    if (n <= 1) return 1;  
    else return n * fact(n-1);  
}  
  
float pi = 3.14;  
int count = 0;
```

Resultado esperado (Saída do Analisador):

None

1(1): /\*...\*/ (COMENTARIO)

2(1): int (PALAVRA RESERVADA int)

2(5): fact (IDENTIFICADOR)

2(9): ( (ABRE PARENTESSES (

2(10): int (PALAVRA RESERVADA int)

2(14): n (IDENTIFICADOR)

2(15): ) (FECHA PARENTESSES ))

2(17): { (ABRE CHAVES {)

3(3): if (PALAVRA RESERVADA if)

3(6): ( (ABRE PARENTESSES (

3(7): n (IDENTIFICADOR)

3(9): <= (MENOR IGUAL <=)

3(12): 1 (NUMERO INTEIRO)

3(13): ) (FECHA PARENTESSES ))

3(15): return (PALAVRA RESERVADA return)

3(22): 1 (NUMERO INTEIRO)

3(23): ; (PONTO E VIRGULA ;)

4(3): else (PALAVRA RESERVADA else)

4(8): return (PALAVRA RESERVADA return)

4(15): n (IDENTIFICADOR)

4(17): \* (VEZES \*)

4(19): fact (IDENTIFICADOR)

4(23): ( (ABRE PARENTESSES (

4(24): n (IDENTIFICADOR)

4(25): - (MENOS -)

4(26): 1 (NUMERO INTEIRO)

4(27): ) (FECHA PARENTESSES ))

4(28): ; (PONTO E VIRGULA ;)

5(1): } (FECHA CHAVES })

7(1): float (PALAVRA RESERVADA float)

7(7): pi (IDENTIFICADOR)

```

7(10): = (ATRIBUICAO =)
7(12): 3.14 (NUMERO REAL)
7(16): ; (PONTO E VIRGULA ;)

8(1): int (PALAVRA RESERVADA int)
8(5): count (IDENTIFICADOR)
8(11): = (ATRIBUICAO =)
8(13): 0 (NUMERO INTEIRO)
8(14): ; (PONTO E VIRGULA ;)

```

Exemplo de Entrada com Erro (teste\_erros.txt):

C/C++

```
int val = 10@;
```

None

```

1(1): int (PALAVRA RESERVADA int)
1(5): val (IDENTIFICADOR)
1(9): = (ATRIBUICAO =)
1(11): 10 (NUMERO INTEIRO)
(1) Erro léxico na linha 1 e na coluna 13. Entrada -> "@"
1(14): ; (PONTO E VIRGULA ;)

```

Os testes demonstraram que o analisador léxico foi capaz de:

- Identificar corretamente todos os tipos de tokens definidos para a linguagem C-.
- Ignorar espaços em branco, tabulações e novas linhas.
- Processar e ignorar corretamente comentários de múltiplas linhas.
- Reportar erros léxicos, indicando o caractere inválido, a linha e a coluna correspondentes. Manter a contagem correta de linhas e colunas durante a análise.

Além da validação do analisador léxico, também foram realizados testes para a **análise sintática**, com o objetivo de verificar se o analisador gerado pelo Bison é capaz de reconhecer corretamente estruturas válidas da linguagem C- e identificar construções



sintaticamente inválidas. O analisador sintático foi integrado ao analisador léxico e executado com arquivos de teste criados especificamente para testar erros comuns de sintaxe.

Os principais tipos de erros testados com suas respectivas saídas encontradas foram:

#### 1. Erro de Fechamento de Parênteses

```
375     $4 = token error ()
376     $5 = nterm comando ()
377 (2) Erro sintático na linha 5, coluna 6: Esperado ')' após condição do while
378 -> $$ = nterm iteracao_decl ()
379 Entering state 81
```

#### 2. Erro em Arrays Multidimensionais

```
171     $5 = token error ()
172     $4 = token CLOSE_BRACKET ()
173 (2) Erro sintático na linha 3, coluna 19: Dimensão inválida no array
174 -> $$ = nterm arranjo_dimensao ()
175 Entering state 55
176 Stack now 0 10 15 18 29 42 47 57 23 35 19 33 46 55
```

#### 3. Erro de Fechamento de Colchetes

```
     $5 = token error ()
     $6 = token SEMICOLON ()
(2) Erro sintático na linha 2, coluna 16: Esperado ']' após índice do array
-> $$ = nterm var_declaracao ()
Entering state 63
```

#### 4. Erro em Condição If/While

```
375     $4 = token error ()
376     $5 = nterm comando ()
377 (2) Erro sintático na linha 5, coluna 6: Esperado ')' após condição do while
378 -> $$ = nterm iteracao_decl ()
379 Entering state 81
```

#### 5. Erro de Fechamento de Chaves

```
476     $2 = nterm local_declaracoes ()
477     $3 = nterm comando_lista ()
478     $4 = token error ()
479 (2) Erro sintático na linha 6, coluna 2: Esperado '}' para fechar bloco de comandos
480 -> $$ = nterm composto_decl ()
481 Entering state 51
```

Os testes de análise sintática confirmaram que:

- **Comandos válidos** são corretamente reconhecidos, como declarações de variáveis, comandos `if`, `while`, `return` e chamadas de função.
- **Erros estruturais** comuns, como falta de parênteses, colchetes ou chaves, são detectados com mensagens apropriadas.
- A **restrição gramatical de arrays unidimensionais** foi respeitada conforme o previsto pela GLC.

Assim, a análise sintática complementa a análise léxica ao garantir que o código segue as regras estruturais da linguagem C-, oferecendo mensagens claras para auxiliar no diagnóstico de erros de programação.

Assim, a análise sintática complementa a análise léxica ao garantir que o código segue as regras estruturais da linguagem C-, oferecendo mensagens claras para auxiliar no diagnóstico de erros de programação.

Por fim, foram executados testes após as mudanças da terceira etapa do projeto

#### Teste Programa Básico:

```
/* Programa básico em C- */
int main() {
    int x;
    int y;

    x = 5;
    y = 10;

    if (x > 0) {
        int z;
        z = 10;
        y = x + 2 - 1;
    }

    while (y < 20) {
        y = y + 1;
    }

    return 0;
}
```

## Tabela de Símbolos E Código Intermediário

| Tabela de Símbolos |      |          |        |       |                 |                |
|--------------------|------|----------|--------|-------|-----------------|----------------|
| Nome               | Tipo | DataType | Escopo | Linha | Endereço(bytes) | Tamanho(bytes) |
| =====              |      |          |        |       |                 |                |
| x                  | var  | int      | 1      | 3     | 4               | 4              |
| y                  | var  | int      | 1      | 4     | 8               | 4              |
| z                  | var  | int      | 2      | 10    | 0               | 4              |
| main               | func | int      | 1      | 2     | 0               | 4              |

  

| Código Intermediário Gerado |  |  |  |  |  |  |
|-----------------------------|--|--|--|--|--|--|
| 1: x = 5                    |  |  |  |  |  |  |
| 2: y = 10                   |  |  |  |  |  |  |
| 3: z = 10                   |  |  |  |  |  |  |
| 4: t0 = x + 2               |  |  |  |  |  |  |
| 5: t1 = t0 - 1              |  |  |  |  |  |  |
| 6: y = t1                   |  |  |  |  |  |  |

## 5. Conclusão

A implementação do analisador léxico para a linguagem C- utilizando a ferramenta Flex foi concluída com sucesso. O analisador é capaz de segmentar o código-fonte em tokens, descartar elementos não relevantes como espaços e comentários, e identificar erros léxicos básicos, fornecendo informações de localização precisas.

Esta etapa representa a base fundamental para as fases subsequentes da compilação, como a **análise sintática**, que recebe a sequência de tokens gerada pelo analisador léxico e verifica se ela obedece às regras gramaticais da linguagem. Para essa segunda fase, foi utilizado o Bison, que, a partir de uma Gramática Livre de Contexto (GLC), gerou automaticamente um analisador sintático capaz de identificar construções válidas e relatar erros de sintaxe com suporte a mensagens de depuração.

A implementação da análise semântica, da tabela de símbolos e do gerador do código de três endereços foi bem sucedida, onde conseguimos aplicar conceitos teóricos na prática e desenvolver o trabalho de acordo com as instruções.

A integração entre Flex e Bison demonstrou-se eficiente, possibilitando a construção de uma ferramenta coesa para análise inicial de programas escritos na linguagem C-.

## 6. Referências Bibliográficas

1. AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 1986. (Livro do Dragão)
2. LEVINE, John R.; MASON, Tony; BROWN, Doug. *lex & yacc*. O'Reilly & Associates, Inc., 1992.
3. Documentação Oficial do Flex: <https://flex.sourceforge.io/manual/>
4. Site oficial do JFLAP: <https://jflap.org/>
5. **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D.** (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
6. HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à teoria de autômatos, linguagens e computação*. 3. ed. São Paulo: Pearson Addison Wesley, 2002.
7. Documentação oficial do Bison: <https://www.gnu.org/software/bison/manual/>.